

Clase 9

DATOS COMPUESTOS

CC1002 Introducción a la
Programación

Datos compuestos

- Problema: sumar fracciones
- Fórmula: $\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}$
- Función que recibe valores **a**, **b**, **c**, **d** y calcula resultado evaluando la fórmula

CC1002 Introducción a la
Programación

Datos compuestos

- Solución usando tipos de datos básicos:

```
# sumaFracciones : int int int int -> float
# calcula la suma entre dos fracciones a/b y c/d
# ejemplo: sumaFracciones(1, 2, 3, 4) devuelve 1.25
def sumaFracciones(a, b, c, d):
    return (a * d + b * c) / b * d

# Test (usa la funcion cerca)
epsilon = 0.000001
assert cerca(sumaFracciones (1, 2, 3, 4), 1.25 , epsilon)
```

- Problema: función no retorna una fracción

CC1002 Introducción a la
Programación

Datos compuestos

- Estructura (**struct**):
 - Tipo de dato que permite encapsular un conjunto fijo de valores (de uno o más tipos)
 - Representado por atributos, que conformar un único valor compuesto
 - Se le denomina **dato compuesto**
- Módulo **estructura**: contiene la función **crear** para definir datos compuestos

CC1002 Introducción a la
Programación

Datos compuestos

- Ejemplo de uso de módulo estructura:

```
import estructura
estructura.crear('nombre', 'atributo1 atributo2 ... atributoN')
```

- Definiendo fracciones como un **struct**:

```
>>> import estructura
>>> estructura.crear('fraccion', 'numerador denominador')
>>> a = fraccion(1, 2)
>>> a
fraccion(numerador=1, denominador=2)
>>> a.numerador
1
>>> a.denominador
2
```

CC1002 Introducción a la
Programación

Datos compuestos

- No se pueden modificar los valores de los atributos:

```
>>> a.numerador = 3
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    a.numerador = 3
AttributeError: can't set attribute
```

- Denominaremos **no mutables** aquellos tipos de datos que no se pueden modificar una vez creados (ej., los *Strings*)

CC1002 Introducción a la
Programación

Datos compuestos

- Receta de diseño con datos compuestos:
 - Reconocer desde el planteamiento del problema las estructuras de datos que se requieran
 - Diseñar la(s) estructura(s) de datos, especificando atributos y sus tipos, por ejemplo
- Seguir receta de diseño usual para las funciones que ocupen estas estructuras

```
# fraccion: numerador (int) denominador (int)
estructura.crear('fraccion', 'numerador denominador')
```

CC1002 Introducción a la
Programación

Datos compuestos

```
import estructura

# fraccion: numerador (int) denominador (int)
estructura.crear('fraccion', 'numerador denominador')

# sumaFracciones: fraccion fraccion -> fraccion
# devuelve la fraccion que resulta al sumar dos fracciones f1 y f2
# ejemplo: sumaFracciones(fraccion(1, 2), fraccion(3, 4))
# devuelve fraccion(10, 8)
def sumaFracciones(f1, f2):
    num = f1.numerador * f2.denominador + f1.denominador * f2.numerador
    den = f1.denominador * f2.denominador
    return fraccion(num, den)

# Tests
f12 = fraccion(1, 2)
f34 = fraccion(3, 4)
assert sumaFracciones(f12, f34) == fraccion(10, 8)
```

CC1002 Introducción a la
Programación

Datos compuestos

- El atributo de un dato compuesto puede ser de tipo de otro dato compuesto

```
# persona: nombre (str) apellido (str)
estructura.crear('persona', 'nombre apellido')

# infoCuenta: codigo (str) saldo (int)
estructura.crear('infoCuenta', 'codigo saldo')

# cliente: usuario (persona) cuenta (infoCuenta)
estructura.crear('cliente', 'usuario cuenta')

personal = persona('Juan', 'Perez')
infocuenta1 = infoCuenta('100-0', 2000)

persona2 = persona('Ana', 'Gonzalez')
infocuenta2 = infoCuenta('105-3', 5000)

cliente1 = cliente(personal, infocuenta1)
cliente2 = cliente(persona2, infocuenta2)
print(cliente1.usuario.apellido) # Muestra en pantalla: Perez
print(cliente2.cuenta.saldo)     # Muestra en pantalla: 5000
```

CC1002 Introducción a la
Programación

Datos compuestos

- Sólo podemos guardar un número finito de atributos en un dato compuesto
- ¿Cómo se podría guardar información si uno no conoce la cantidad de datos a priori?
 - Crear un dato con X atributos: no nos sirve si cantidad de datos es $> X$
 - Usar un valor de X muy grande: se podría estar desperdiciando mucha memoria

CC1002 Introducción a la
Programación

Datos compuestos

- Pero, podríamos definir el siguiente dato compuesto, que tiene dos atributos:

lista: valor (any = cualquier tipo) siguiente (**lista**)

CC1002 Introducción a la
Programación

Para la próxima clase

- Leer los Capítulos 9.1, 9.2 y 9.3 del apunte

CC1002 Introducción a la
Programación